Backend - Laravel

Model

Rostagni Csaba

2025. november 17.

Tartalom

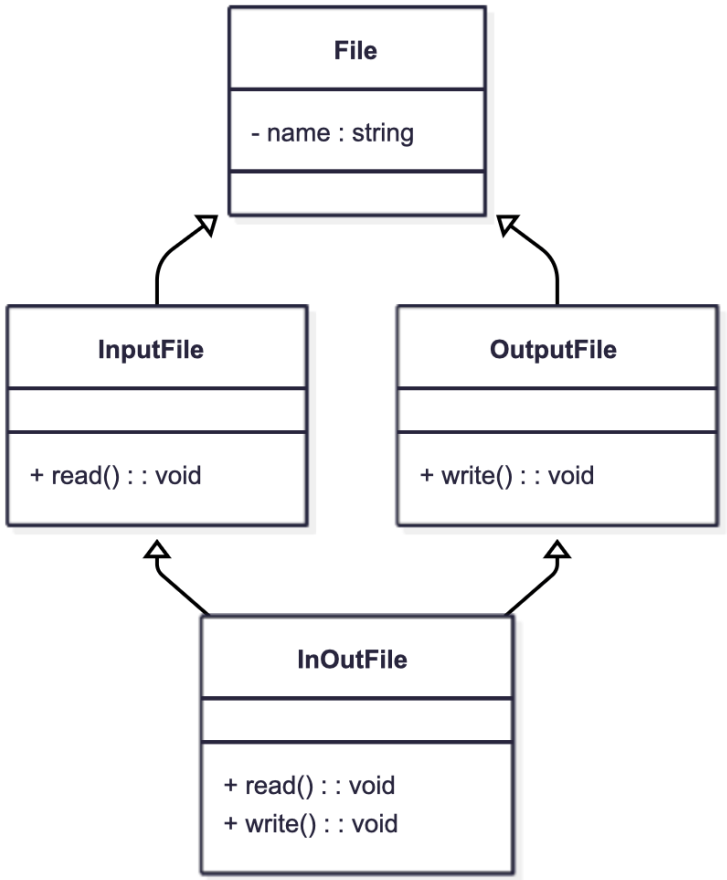
1. Factory
2. Faker

Tartalom

1. Factory
   - Trait
   - Bevezető

Többszörös öröklődés

- Bizonyos nyelvek támogatják a többszörös öröklődést
  - Egy osztály örökölhet egyszerre több osztálytól
  - Kódismétlés csökkenthető
  - Melyik függvény érvényesül, ha azonos a nevük mindkét szülőben?
  - **PHP NEM támogatja a többszörös öröklődést**



Trait

- PHP-ban a Trait-ek a kódújrahasznosítást teszik lehetővé
- Az így megvalósított metódus több, egymástól független osztályban is használható
- Egy osztály több Trait-et is haszánlhat
- A Trait mini osztály jellegű
- Nem lehet példányosítani
- Tartalmazhat metódust akár többet is
- Tartalmazhat property-t is, nem ütközhet más property-vel

PHP

```php
trait Osszeadas {
    public function osszead(int $a, int $b) {
        return $a + $b;
    }
}
```

Linkek:

- Traits - PHP dokumentáció

Trait példa

PHP

```php
trait Osszeadas {
    public function osszead(int $a, int $b) {
        return $a + $b;
    }
}
class Penztar {
    int $a; int $b;
    use Osszeadas;
    public function __construct(int $a, int $b) {
        $this->a = $a;
        $this->b = $b;
    }
    public function fizetendo() : int {
        return $this->osszead($this->a,$this->b);
    }
}
$penztar = new Penztar(5, 7);
echo $penztar->fizetendo(); // 12
echo $penztar->osszead(19,3); // 22
```

Tartalom

1. Factory
   - Trait
   - Bevezető

Factory

- Teszteléshez szükség lehet nagy mennyiségű tesztadatra
- A PHP Faker-t használja
  - A `fake()` segédfüggvény adja vissza a Faker objektumot
  - A nyelv beállítása az `.env` fájlban: `APP_FAKER_LOCALE=hu_HU`
- A modellel együttműködve feltölti az adatbázist
- A fájlok a `database/factories` mappában vannak

Linkek:

- Factories - Laravel dokumentáció
- PHP Faker

Factory generálása

PHP
```
php artisan make:factory CarFactory
```

- A `Car` osztályhoz tartozó factory fájl a `CarFactory` lesz
  - Osztály neve + "Factory"
  - Legenerált fájl: `database/factories/CarFactory.php`

PHP
```php
namespace Database\Factories;

use Illuminate\Database\Eloquent\Factories\Factory;

class CarFactory extends Factory
{
    public function definition(): array
    {
        return [ ];
    }
}
```

Factory generálása

```php
php artisan make:model -f Car
```

- Model generálásával együtt létrehozható factory is
- A `-f` kapcsoló miatt létrehozza a `CarFactory`-t is, ha a modell még nem létezik

```php
<?php namespace App\Models;
use Illuminate\Database\Eloquent\Factories\HasFactory;
use Illuminate\Database\Eloquent\Model;

class Car extends Model
{
    use HasFactory;
}
```

- Az így generált modell már tartalmazza osztályon belül a `use HasFactory`-t is, ami **feltétel szükséges a működéshez**

Tartalom

1. Factory
2. Faker

Tartalom

2 Faker
- Szövegek
  - words()
- Számok
  - randomNumber()
  - randomFloat()
- Speciális
  - randomElement()
  - regexify()
- Módosítók
- Példa

words()

PHP
```php
fake()->words(3)
```

- Megadott számú szót generál

Eredmény
```
['praesentium', 'possimus', 'modi']
```

PHP
```php
fake()->words(3, true)
```

- Ha a második paramétere true, akkor tömb helyett szöveget ad vissza

Eredmény
```
'praesentium possimus modi'
```

Linkek:

- words() - PHP faker

Tartalom

2 Faker
- Szövegek
  - words()
- Számok
  - randomNumber()
  - randomFloat()
- Speciális
  - randomElement()
  - regexify()
- Módosítók
- Példa

Számok generálása

- `randomDigit()` Véletlen számjegy
- `randomNumber()` megadott számjegyből álló egész szám
- `numberBetween()` két szám közötti egész számot ad
- `randomFloat()` valós számot generál
- ...

randomNumber()

PHP
```php
fake()->randomNumber(3)
```

- 0 és 3 közötti számjegyű számot generál

Eredmény
```
111 vagy 5 vagy 12 vagy 999
```

PHP
```php
fake()->randomNumber(4, true)
```

- Pontosan 4 számjegyű számokat ad

Eredmény
```
4657 vagy 9999 vagy 6681
```

Linkek:

- randomNumber() - PHP faker

randomFloat()

PHP
```php
fake()->randomFloat(3)
```

- Az első paraméter a tizedesek számát határozza meg

Eredmény
```
837.622 vagy 112.956 vagy 985.112
```

PHP
```php
fake()->randomFloat(3, 1.888, 22)
```

- A második paraméter a minimális érték
- A harmadik paraméter a maximális érték

Eredmény
```
1.888 vagy 16.334 vagy 21.765
```

Linkek:

- randomFloat() - PHP faker

Tartalom

2 Faker
- Szövegek
  - words()
- Számok
  - randomNumber()
  - randomFloat()
- Speciális
  - randomElement()
  - regexify()
- Módosítók
- Példa

randomElement()

PHP
```php
fake()->randomElement(['A', 'B', 'C', 'D'])
```

- Véletlenszerűen választ **egy** elemet a megadott tömbből

Eredmény
```
C
```

Linkek:

- randomElement() - PHP faker

regexify()

PHP
```php
fake()->regexify('[A-Z]{2}-[A-Z]{2}-[0-9]{3}')
```

- A regexnek megfelelő értéket generál
- A fenti példa nem tökéletes, de az új szabványos rendszámra hasonlító értékeket ad

Eredmény
```
AA-DX-729
```

Tartalom

2 Faker
- Szövegek
  - words()
- Számok
  - randomNumber()
  - randomFloat()
- Speciális
  - randomElement()
  - regexify()
- Módosítók
- Példa

optional()

```php
fake()->optional()->email()
```

- Az `optional()` módosító miatt néhány email cím helyett NULL értéket ad

```php
fake()->optional(0.25)->email()
```

- Az email 75 százaléka NULL érték lesz

```php
fake()->optional(0.75)->email()
```

- Az email 25 százaléka NULL érték lesz

Tartalom

2 Faker
- Szövegek
  - words()
- Számok
  - randomNumber()
  - randomFloat()
- Speciális
  - randomElement()
  - regexify()
- Módosítók
- Példa

Autók példa: Car (modell)

PHP

```php
<?php

namespace App\Models;

use Illuminate\Database\Eloquent\Factories\HasFactory;
use Illuminate\Database\Eloquent\Model;

class Car extends Model
{
    use HasFactory;
}
```

- A `UseHasFactory` Trait használata elengedhetetlen!

Autók példa: CarFactory

PHP

```php
<?php
namespace Database\Factories;

use Illuminate\Database\Eloquent\Factories\Factory;

class CarFactory extends Factory
{
  public function definition(): array
  {
    $manufacturers = ["Opel", "Ford", "Honda", "Audi", "Skoda"];
    return [
      "plate_number" => fake()->regexify('[A-Z]{2}-[A-Z]{2}-[0-9]{3}'),
      "manufacturer_id" => fake()->randomElement($manufacturers),
      "model" => ucfirst(fake()->word()),
      "color" => fake()->randomElement(["piros","kék","ezüst"]),
      "consumption" => fake()->randomFloat(2,4,14),
      "category" => "M1",
      "fuel_type" => "benzin",
    ];
  }
}
```

Autók példa: CarSeeder

PHP

```php
<?php
namespace Database\Seeders;

use App\Models\Car;

class CarSeeder extends Seeder
{
  Car::factory(10)->create();

  Car::factory(10)->create([
      "manufacturer_id" => 1
  ]);
}
```

- A `factory()` függvényben megadott darabszámoú elemet hoz létre
- A `create()` kaphat egy tömböt, ami felülírja a definícióban megadott értékeket